

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №2
По предмету: “Вычислительная математика”
Вариант № 13**

Выполнил: Федоров Дмитрий
Александрович Р3206
Проверил: Зинченко А. А.
г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
1.	3
2.	3
3.	3
4.	4
Исходный код программы:	4
Результаты работы программы:	5
Заключение:	7

ЗАДАНИЕ:

Лабораторная работа №2 Численное решение нелинейных уравнений и систем

Цель работы: изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов.

№ варианта определяется как номер в списке группы согласно ИСУ.

Лабораторная работа состоит из двух частей: вычислительной и программной.

1 Вычислительная реализация задачи:

Состоит из двух частей и отражается ТОЛЬКО в отчете.

1 часть. Решение нелинейного уравнения

Задание:

1. Отделить корни заданного нелинейного уравнения графически (вид уравнения представлен в табл. 6)
2. Определить интервалы изоляции корней.
3. Уточнить корни нелинейного уравнения (см. табл. 6) с точностью $\varepsilon=10^{-2}$.
4. Используемые методы для уточнения каждого из 3-х корней многочлена представлены в таблице 7.
5. Вычисления оформить в виде таблиц (1-5), в зависимости от заданного метода. Для всех значений в таблице удерживать **3 знака** после запятой.
 - 5.1 Для метода половинного деления заполнить таблицу 1.
 - 5.2 Для метода хорд заполнить таблицу 2.
 - 5.3 Для метода Ньютона заполнить таблицу 3.
 - 5.4 Для метода секущих заполнить таблицу 4.
 - 5.5 Для метода простой итерации заполнить таблицу 5. **Проверить условие сходимости метода на выбранном интервале.**
6. Заполненные таблицы отобразить в отчете.

2 часть. Решение системы нелинейных уравнений

Задание:

1. Отделить корни заданной системы нелинейных уравнений графически (вид системы представлен в табл. 8).
2. Используя указанный метод, решить систему нелинейных уравнений с точностью до 0,01.
3. Для метода простой итерации проверить условие сходимости метода.
4. Подробные вычисления привести в отчете.

Таблица 8

2 Программная реализация задачи:

Для нелинейных уравнений:

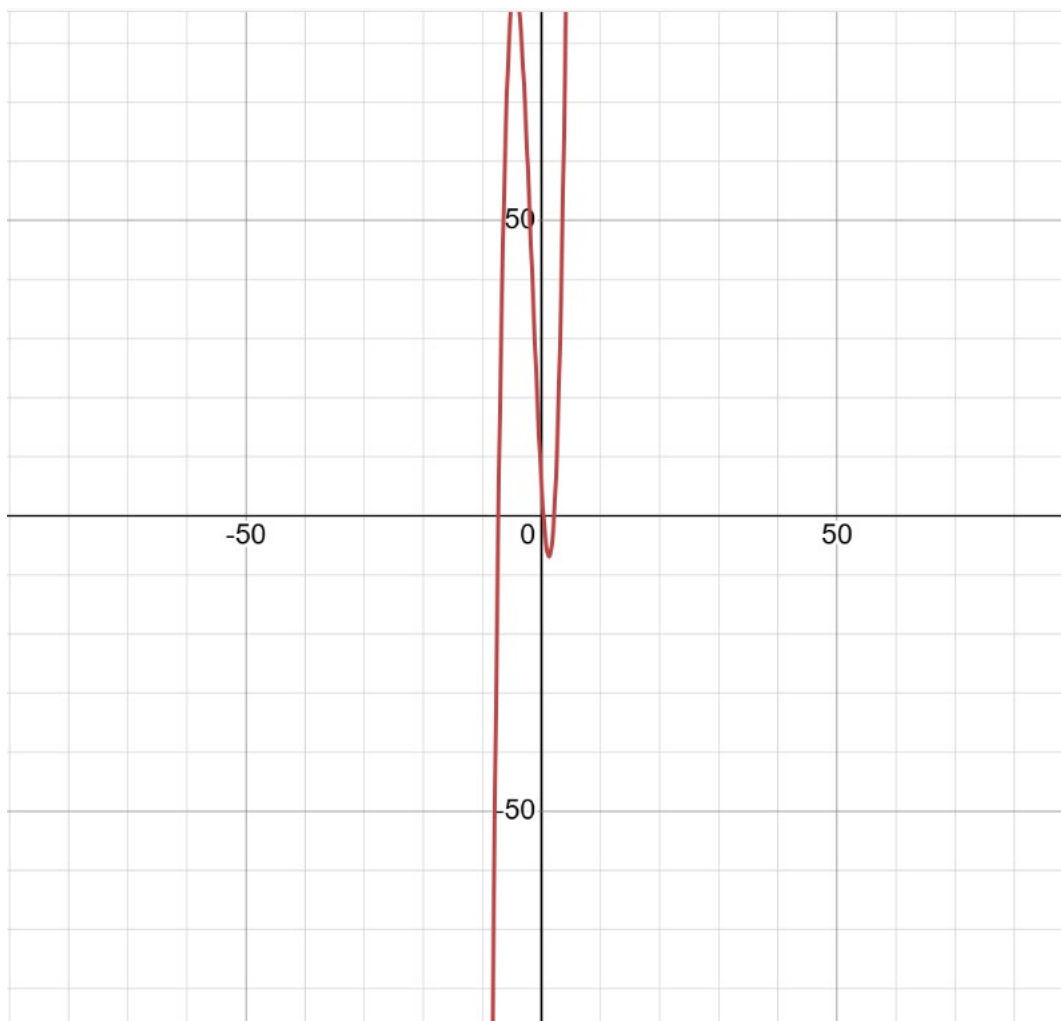
1. Все численные методы (см. табл. 9) должны быть реализованы в виде отдельных подпрограмм/методов/классов.
2. Пользователь выбирает уравнение, корень/корни которого требуется вычислить (3-5 функций, в том числе и трансцендентные), из тех, которые предлагает программа.
3. Предусмотреть ввод исходных данных (границы интервала/начальное приближение к корню и погрешность вычисления) из файла или с клавиатуры по выбору конечного пользователя.
4. Выполнить верификацию исходных данных. Необходимо анализировать наличие корня на введенном интервале. Если на интервале несколько корней или они отсутствуют – выдавать соответствующее сообщение. Программа должна реагировать на некорректные введенные данные.
5. Для методов, требующих начальное приближение к корню (методы Ньютона, секущих, хорд с фиксированным концом, простой итерации), выбор начального приближения x_0 (a или b) вычислять в программе.
6. Для метода простой итерации проверять достаточное условие сходимости метода на введенном интервале.
7. Предусмотреть вывод результатов (найденный корень уравнения, значение функции в корне, число итераций) в файл или на экран по выбору конечного пользователя.
8. Организовать вывод графика функции, график должен полностью отображать весь исследуемый интервал.

Для систем нелинейных уравнений:

1. Пользователь выбирает предлагаемые программой системы двух нелинейных уравнений (2-3 системы).
2. Организовать вывод графика функций.
3. Начальные приближения ввести с клавиатуры.
4. Для метода простой итерации проверить достаточное условие сходимости.
5. Организовать вывод вектора неизвестных: x_1, x_2 .
6. Организовать вывод количества итераций, за которое было найдено решение.
7. Организовать вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$
8. Проверить правильность решения системы нелинейных уравнений.

Часть 1

Уравнение: $x^3 + 4,81x^2 - 17,37x + 5,38$



Интервалы изоляции корней:

$$x_1 \in [-8; -7]$$

$$x_2 \in [0; 1]$$

$$x_3 \in [2; 3]$$

Уточнение корня уравнения методом хорд

№ шага	a	b	x	f(a)	f(b)	f(x)	$ x_{k+1} - x_k $
--------	---	---	---	------	------	------	-------------------

0	-8	-7	-7.247	-59.82	19.659	3.246	
1	-7.247	-7	-7.296	3.271	19.659	-0.244	0.049
2	-7.296	-7	-7.292	-0.222	19.659	0.016	0.004

шаг 0: $[a;b] = [-8;-7]$

шаг 1: $f(b) \cdot f(x_0) < 0 \Rightarrow [a;b] = [-7.247; -7]$

шаг 2: $f(b) \cdot f(x_0) < 0 \Rightarrow [a;b] = [-7.296; -7]$

Уточнение корня уравнения методом Ньютона

№ итерации	x_k	$f(x_k)$	$f'(x_k)$	x_{k+1}	$ x_{k+1} - x_k $
1	0.5	-1.977	-11.81	0.332	0.168
2	0.332	0.179	-13.845	0.344	0,002

Уточнение корня уравнения методом простой итерации

№ итерации	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	2.5	2.159	0.362	0.341
2	2.159	2.141	0.061	0.018
3	2.141	2.138	0.011	0.003

$$x^3 + 4,81x^2 - 17,37x + 5,38 = 0$$

$$\varphi(x) = \sqrt{\frac{(-x^3 + 17.37x - 5.38)}{4.81}}$$

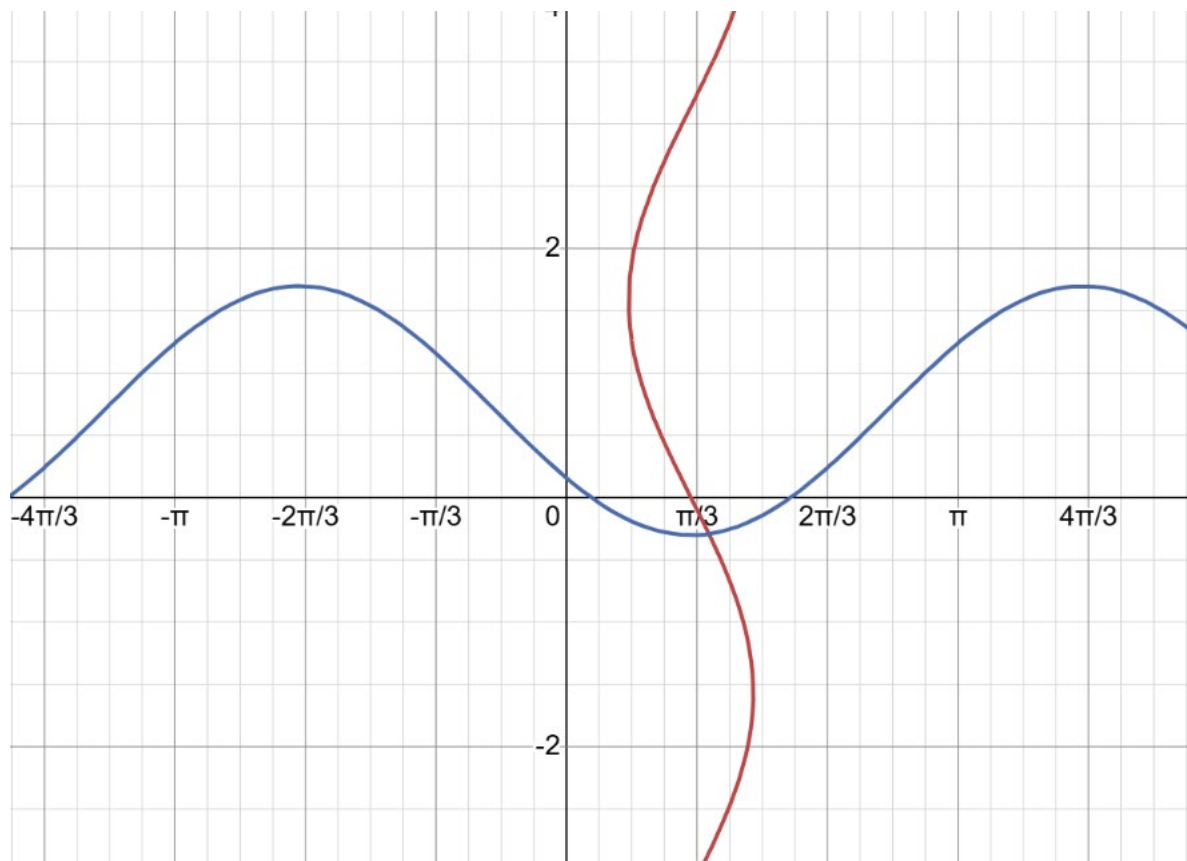
$$\varphi'(x) = \frac{-3x^2 + 17.37}{2 * 4.81 * \varphi(x)}$$

$$q = \max|\varphi'(x)| = 0.5 \text{ (при } x = 3) \Rightarrow \text{условие сходимости выполнено}$$

Часть 2

$$\begin{cases} \sin(y) + 2x = 2 \\ y + \cos(x - 1) = 0,7 \end{cases}$$

Метод простых итераций



$$\begin{cases} \sin(y) + 2x = 2 \\ y + \cos(x - 1) = 0,7 \end{cases}$$

$$\begin{cases} x = \varphi_1(x, y) \\ y = \varphi(x, y) \end{cases}$$

$$\begin{cases} x_{k+1} = 1 - \frac{\sin(y_k)}{2} \\ y_{k+1} = 0.7 - \cos(x_k - 1) \end{cases}$$

Начальное приближение:

$$x_0 = 1.2; y_0 = -0.3;$$

$$x \in [1.1; 1.3]; y \in [-0.35; -0.25]$$

Проверка сходимости:

Частные производные:

$$\frac{d\varphi_1}{dx} = 0; \frac{d\varphi_1}{dy} = -\frac{\cos(y)}{2}$$

$$\frac{d\varphi_2}{dy} = \sin(x-1); \frac{d\varphi_2}{dx} = 0$$

$$1) \left| \frac{d\varphi_1}{dx} \right| + \left| \frac{d\varphi_1}{dy} \right| = \left| \frac{\cos(y)}{2} \right|$$

$$\frac{\cos(-0,35)}{2} = 0.469$$

$$\frac{\cos(-0,25)}{2} = 0.48$$

$$\max \left| \frac{\cos(y)}{2} \right| = 0.48 < 1$$

$$2) \left| \frac{d\varphi_2}{dx} \right| + \left| \frac{d\varphi_2}{dy} \right| = |\sin(x-1)|$$

$$|\sin(0.1)| = 0.099$$

$$|\sin(0.2)| = 0.198$$

$$\max |\sin(x-1)| = 0.198 < 1$$

Условия сходимости выполнены

Итерация 1:

$$x_1 = 1 - \frac{\sin(-0,300)}{2} = 1 + 0,148 = 1,148$$

$$y_1 = 0,7 - \cos(1,200 - 1) = 0,7 - 0,980 = -0,280$$

$$|x_{k+1} - x_k| = |1,148 - 1,200| = 0,052$$

$$|y_{k+1} - y_k| = |-0,280 + 0,300| = 0,020$$

Итерация 2:

$$x_2 = 1 - \sin(-0,280)/2 = 1 + 0,138 = 1,138$$

$$y_2 = 0,7 - \cos(1,148 - 1) = 0,7 - 0,989 = -0,289$$

$$|x_{k+1} - x_k| = |1,138 - 1,148| = 0,010$$

$$|y_{k+1} - y_k| = |-0,289 + 0,280| = 0,009$$

Итерация 3:

$$x_3 = 1 - \frac{\sin(-0,289)}{2} = 1 - \frac{-0,285}{2} = 1 + 0,143 = 1,143$$

$$y_3 = 0,7 - \cos(1,138 - 1) = 0,7 - 0,990 = -0,290$$

$$|x_{k+1} - x_k| = |1,143 - 1,138| = 0,005$$

$$|y_{k+1} - y_k| = |-0,290 - (-0,289)| = 0,001$$

$$x = 1,143 \quad y = -0,290$$

Задание 2

Листинг программы:

```
import math
import matplotlib.pyplot as plt
import numpy as np

equations = [
    {
        'name': 'x^3 + 4.81x^2 - 17.37x + 5.38',
        'f': lambda x: x**3 + 4.81*x**2 - 17.37*x + 5.38,
        'df': lambda x: 3*x**2 + 9.62*x - 17.37,
        'phi': lambda x: math.sqrt((-x**3 + 17.37*x - 5.38) / 4.81),
    },
    {
        'name': 'sin(x) - 0.5',
        'f': lambda x: math.sin(x) - 0.5,
        'df': lambda x: math.cos(x),
        'phi': lambda x: math.asin(0.5)
    },
    {
        'name': 'e^x - 3x',
        'f': lambda x: math.exp(x) - 3*x,
        'df': lambda x: math.exp(x) - 3,
        'phi': lambda x: math.log(3*x)
    },
    {
        'name': 'x^2 - 2',
        'f': lambda x: x**2 - 2,
        'df': lambda x: 2*x,
        'phi': lambda x: math.sqrt(2)
    },
    {
        'name': 'cos(x) - x',
        'f': lambda x: math.cos(x) - x,
        'df': lambda x: -math.sin(x) - 1,
        'phi': lambda x: math.cos(x)
    }
]
```

```

systems = [
    {
        'name': 'sin(y) + 2x = 2; y + cos(x-1) = 0.7',
        'F1': lambda x, y: np.sin(y) + 2*x - 2,
        'F2': lambda x, y: y + np.cos(x-1) - 0.7,
        'dF1dx': lambda x, y: 2,
        'dF1dy': lambda x, y: np.cos(y),
        'dF2dx': lambda x, y: -np.sin(x-1),
        'dF2dy': lambda x, y: 1
    },
    {
        'name': 'x^2 + y^2 = 4; y = x^2',
        'F1': lambda x, y: x**2 + y**2 - 4,
        'F2': lambda x, y: y - x**2,
        'dF1dx': lambda x, y: 2*x,
        'dF1dy': lambda x, y: 2*y,
        'dF2dx': lambda x, y: -2*x,
        'dF2dy': lambda x, y: 1
    }
]

```

```

def read_from_file(filename="data.txt"):

```

```

    try:
        with open(filename, 'r') as f:
            lines = f.readlines()
            a = float(lines[0].strip())
            b = float(lines[1].strip())
            eps = float(lines[2].strip())
            return a, b, eps

```

```

    except:
        print("Ошибка чтения файла")
        return None, None, None

```

```

def check_interval(f, a, b, num_points=100):

```

```

    fa = f(a)
    fb = f(b)

```

```

    if fa * fb > 0:

```

```

        print("Ошибка: функция имеет одинаковые знаки на концах интервала, на интервале
        либо нет корней, либо их чётное количество")
        return False

```

```

    crossings = 0
    step = (b - a) / num_points
    x_prev = a
    f_prev = fa

```

```

    for i in range(1, num_points + 1):

```

```

        x_curr = a + i * step
        f_curr = f(x_curr)
        if f_prev * f_curr < 0:
            crossings += 1

```

```

    x_prev, f_prev = x_curr, f_curr

    if crossings != 1:
        print(f"Ошибка: на интервале {crossings} корней")
        return False

    return True

def choose_initial(f, a, b):
    fa = abs(f(a))
    fb = abs(f(b))
    return a if fa < fb else b

def output_result(result, to_file=False, filename="result.txt"):
    if to_file:
        with open(filename, 'w') as f:
            f.write(str(result))
        print(f"Результат сохранён в {filename}")
    else:
        print("Результат\n:")
        print(result)

def plot_function(eq, a, b):
    x = np.linspace(a, b, 400)
    y = [eq['f'](xi) for xi in x]

    plt.figure(figsize=(10, 6))
    plt.plot(x, y, 'b-', linewidth=2, label=eq['name'])
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.grid(True, alpha=0.3)
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.title(f'График функции: {eq["name"]}')
    plt.show()

def plot_system(system, x_range, y_range):
    x = np.linspace(x_range[0], x_range[1], 100)
    y = np.linspace(y_range[0], y_range[1], 100)
    X, Y = np.meshgrid(x, y)

    Z1 = system['F1'](X, Y)
    Z2 = system['F2'](X, Y)

    plt.figure(figsize=(10, 8))
    contour1 = plt.contour(X, Y, Z1, levels=[0], colors='blue', linewidths=2)
    contour2 = plt.contour(X, Y, Z2, levels=[0], colors='red', linewidths=2)

    plt.xlabel('x')
    plt.ylabel('y')
    plt.title(f'График системы: {system["name"]}')
    plt.grid(True, alpha=0.3)

```

```

plt.show()

def half_division_method(eq, a, b, eps, max_iter=100):
    print("\n" + f"{'№ шага':<8} {'a':<10} {'b':<10} {'x':<10} {'f(x)':<10} {'|a-b|':<10}")

    fa = eq['f'](a)
    fb = eq['f'](b)

    if fa * fb > 0:
        return None, 0, "На интервале нет корня или их несколько"

    iter_count = 0
    while (b - a) > eps and iter_count < max_iter:
        c = (a + b) / 2
        fc = eq['f'](c)

        print("\n" + f"{'iter_count+1':<8} {'a':<10.3f} {'b':<10.3f} {'c':<10.3f} {'fc':<10.3f} {'abs(a-  
b):<10.5f}")

        if abs(fc) < eps:
            return c, iter_count+1, "OK"

        if fa * fc < 0:
            b = c
            fb = fc
        else:
            a = c
            fa = fc

        iter_count += 1

    x = (a + b) / 2
    print("\n" + f"{'iter_count+1':<8} {'a':<10.3f} {'b':<10.3f} {'x':<10.3f} {'eq['f'](x)':<10.3f} {'abs(a-  
b):<10.5f}")
    return x, iter_count+1, "OK"

def secant_method(eq, a, b, eps, max_iter=100):

    print("\n" + f"{'№ итер.':<8} {'x_k-1':<10} {'x_k':<10} {'x_k+1':<10} {'f(x_k+1)':<10}  
{'|x_k+1-x_k|':<10}")

    x0 = choose_initial(eq['f'], a, b)
    x1 = x0 + 2 * eps

    if x1 > b:
        x1 = x0 - 2 * eps
    if x1 < a:
        x1 = x0 + 2 * eps

    x_prev, x_curr = x0, x1

    for i in range(max_iter):

```

```

f_prev = eq['f'](x_prev)
f_curr = eq['f'](x_curr)

if abs(f_curr - f_prev) < 1e-15:
    x_curr = x_prev + eps
    f_curr = eq['f'](x_curr)
    if abs(f_curr - f_prev) < 1e-15:
        return None, i,

x_next = x_curr - f_curr * (x_curr - x_prev) / (f_curr - f_prev)
f_next = eq['f'](x_next)
diff = abs(x_next - x_curr)

print("\n" + f"{i+1:<8} {x_prev:<10.3f} {x_curr:<10.3f} {x_next:<10.3f} {f_next:<10.3f}
{diff:<10.5f}")

if diff < eps:
    return x_next, i+1, "OK"

x_prev, x_curr = x_curr, x_next

return x_curr, max_iter, "Достигнут лимит итераций"

def simple_iteration_method(eq, a, b, eps, max_iter=100):

    if 'phi' not in eq:
        return None, 0, "Не задана функция  $\phi(x)$ "

    q_max = 0
    for x_test in [a, (a+b)/2, b]:
        try:
            dphi = (eq['phi'](x_test + 1e-5) - eq['phi'](x_test)) / 1e-5
            q_max = max(q_max, abs(dphi))
        except:
            pass

    print(f"q =  $|\phi'(x)|$  = {q_max:.3f}")
    if q_max >= 1:
        print("Условие сходимости не выполняется")
        return None, 0, "Условие сходимости не выполняется"
    else:
        print("Условие сходимости выполняется")

    print("\n" + f"{№ итер.:<8} {x_k:<10} {x_{k+1}:<10} {f(x_{k+1}):<10} {|\Delta x|:<10}")

    x = choose_initial(eq['f'], a, b)
    print(f"{0:<8} {x:<10.3f} {'-':<10} {'-':<10} {'-':<10}")

    for i in range(max_iter):
        try:
            x_next = eq['phi'](x)
        except:

```

```

    return None, i, "Ошибка вычисления  $\varphi(x)$ "

    f_next = eq['f'](x_next)
    diff = abs(x_next - x)

    print(f"{i+1:<8} {x:<10.3f} {x_next:<10.3f} {f_next:<10.3f} {diff:<10.5f}")

    if diff < eps:
        return x_next, i+1, "OK"

    x = x_next

    return x, max_iter, "Достигнут лимит итераций"

def newton_system(system, x0, y0, eps, max_iter=100):

    print("\n" + f"{ '№ итер.':<8} { 'x_k':<10} { 'y_k':<10} { 'Δx':<8} { 'Δy':<8} { 'F1':<10} { 'F2':<10}")

    x, y = x0, y0
    F1 = system['F1'](x, y)
    F2 = system['F2'](x, y)
    print(f"{0:<8} {x:<10.3f} {y:<10.3f} {'-':<8} {'-':<8} {F1:<10.3f} {F2:<10.5f}")

    for i in range(max_iter):
        F1 = system['F1'](x, y)
        F2 = system['F2'](x, y)

        if abs(F1) < eps and abs(F2) < eps:
            return (x, y), i, (0, 0), "OK"

        J11 = system['dF1dx'](x, y)
        J12 = system['dF1dy'](x, y)
        J21 = system['dF2dx'](x, y)
        J22 = system['dF2dy'](x, y)

        det = J11 * J22 - J12 * J21

        if abs(det) < 1e-15:
            return None, i, "Определитель близок к нулю"

        dx = (-F1 * J22 + F2 * J12) / det
        dy = (-J11 * F2 + J21 * F1) / det

        x_new = x + dx
        y_new = y + dy

        dx_abs = abs(dx)
        dy_abs = abs(dy)

        F1_new = system['F1'](x_new, y_new)
        F2_new = system['F2'](x_new, y_new)

```

```

    print(f"{i+1:<8} {x_new:<10.3f} {y_new:<10.3f} {dx_abs:<8.3f} {dy_abs:<8.3f}
    {F1_new:<10.3f} {F2_new:<10.5f}")

    if max(dx_abs, dy_abs) < eps:
        return (x_new, y_new), i+1, (dx_abs, dy_abs), "OK"

    x, y = x_new, y_new

    return (x, y), max_iter, (dx_abs, dy_abs), "достигнут лимит итераций"

def main():
    while True:
        print("1 - решение нелинейного уравнения")
        print("2 - решение системы нелинейных уравнений")
        print("0 - выход")

        choice = input("\nваш выбор: ")

        if choice == "0":
            break

        elif choice == "1":
            print("доступные уравнения:")
            for i, eq in enumerate(equations):
                print(f"{i+1}. {eq['name']}")

            try:
                eq_idx = int(input("выберите номер уравнения: ")) - 1
                if eq_idx < 0 or eq_idx >= len(equations):
                    print("Уравнения с таким номером нет")
                    continue

                eq = equations[eq_idx]

                source = input("Ввести данные с клавиатуры (k) или из файла (f)? ")

                if source.lower() == 'f':
                    a, b, eps = read_from_file()
                    if a is None:
                        continue
                else:
                    a = float(input("Левая граница a: "))
                    b = float(input("Правая граница b: "))
                    eps = float(input("Точность например 0.01: "))

                if not check_interval(eq['f'], a, b):
                    print("Попробуйте другой интервал")
                    continue

            print("\nДоступные методы:")
            print("1 - Метод половинного деления")

```

```

print("4 - Метод секущих")
print("5 - Метод простой итерации")

method = input("Выберите метод: ")

result = None
iterations = 0
status = ""

if method == "1":
    result, iterations, status = half_division_method(eq, a, b, eps)
elif method == "4":
    result, iterations, status = secant_method(eq, a, b, eps)
elif method == "5":
    result, iterations, status = simple_iteration_method(eq, a, b, eps)
else:
    print("Неверный метод")
    continue

if result is not None:
    method_names = {"1": "половинного деления", "4": "секущих", "5": "простой
итерации"}
    output_text = f"""
    Уравнение: {eq['name']}
    Метод: {method_names[method]}
    Интервал: [{a}, {b}]
    Точность: {eps}

    Найденный корень: x = {result:.3f}
    Значение функции: f(x) = {eq['f'](result):.6f}
    Число итераций: {iterations}
    Статус: {status}
    """
    print(output_text)

    save = input("Сохранить результат в файл? y/n: ")
    if save.lower() == 'y':
        output_result(output_text, True)

    plot = input("Построить график? y/n: ")
    if plot.lower() == 'y':
        plot_function(eq, a, b)
    else:
        print(f"Ошибка: {status}")

except Exception as e:
    print(f"Ошибка: {e}")

elif choice == "2":
    print("Доступные системы:")
    for i, sys in enumerate(systems):
        print(f"{i+1}. {sys['name']}")

```



```

try:
    sys_idx = int(input("Выберите номер системы: ")) - 1
    if sys_idx < 0 or sys_idx >= len(systems):
        print("Систем с таким номером нет")
        continue

    system = systems[sys_idx]

    x0 = float(input("Начальное приближение x0: "))
    y0 = float(input("Начальное приближение y0: "))
    eps = float(input("Точность (например 0.01): "))

    print("\nДоступные методы:")
    print("6 - Метод Ньютона")

    method = input("Выберите номер метода: ")

    result = None
    iterations = 0
    errors = None
    status = ""

    if method == "6":
        result, iterations, errors, status = newton_system(system, x0, y0, eps)
    else:
        print("Неверный номер метода")
        continue

    if result is not None:
        x, y = result
        dx, dy = errors if errors else (0, 0)

        output_text = f"""
        Система: {system['name']}
        Метод: Ньютона
        Начальное приближение: x0 = {x0}, y0 = {y0}
        Точность: {eps}

        Найденное решение:
        x = {x:.3f}
        y = {y:.3f}

        Проверка:
        F1(x,y) = {system['F1'](x, y):.6f}
        F2(x,y) = {system['F2'](x, y):.6f}

        Погрешности на последней итерации:
        |x_k+1 - x_k| = {dx:.8f}
        |y_k+1 - y_k| = {dy:.8f}

        Число итераций: {iterations}

```

```

        Статус: {status}
        """
    print(output_text)

    save = input("Сохранить результат в файл? y/n: ")
    if save.lower() == 'y':
        output_result(output_text, True)

    plot = input("Построить график системы? y/n: ")
    if plot.lower() == 'y':
        x_range = [x0 - 2, x0 + 2]
        y_range = [y0 - 2, y0 + 2]
        plot_system(system, x_range, y_range)
    else:
        print(f"Ошибка: {status}")

except Exception as e:
    print(f"Ошибка: {e}")

if __name__ == "__main__":
    main()

```

Вывод

В данной лабораторной работе я изучил методы решения нелинейных уравнений и систем. В вычислительной части были уточнены корни нелинейного уравнения с помощью методов хорд, ньютона и простой итерации.

Для системы нелинейных уравнений были уточнены корни методом простой итерации.

Также была написана программа для уточнения корней нелинейных уравнений и систем.